

MANUAL DE INSTALACIÓN

RESTIFY

Jaime Canay Agho

Contenido

1. REQUISITOS PREVIOS	2
2. INSTALACIÓN Y DESPLIEGUE	2
1. CLONAR EL REPOSITORIO.....	2
2. CREAR EL ARCHIVO DE VARIABLES DE ENTORNO	3
3. CONFIGURAR LOS CERTIFICADOS SSL (Opcional para Local)	3
4. LIMPIAR EL ENTORNO DOCKER (Recomendado)	4
5. CONSTRUCCIÓN Y LANZAMIENTO	4
3. VERIFICACIÓN DEL ESTADO	4
PRUEBA DE DISPONIBILIDAD.....	4

Este manual describe los requisitos y pasos necesarios para clonar, configurar y desplegar la plataforma **Restify** en un entorno local o de producción (VPS) utilizando Docker y Nginx como proxy inverso seguro.

1. REQUISITOS PREVIOS

Antes de comenzar, es necesario asegurarse de que el siguiente software está instalado en la máquina de destino:

- **Git:** Para la clonación del repositorio.
- **Docker Desktop** (en Windows/macOS) o **Docker Engine + Docker Compose** (en Linux/Ubuntu).
- **Puertos Libres:** Los puertos 80 (HTTP), 443 (HTTPS) y 3306 (MySQL, si se desea exponer) deben estar disponibles en la máquina host.

2. INSTALACIÓN Y DESPLIEGUE

1. CLONAR EL REPOSITORIO

En primer lugar, se debe abrir la terminal y clonar el repositorio de GitHub en la máquina de destino:

```
git clone https://github.com/jCanay/restify
cd restify
```

Si el proyecto ya ha sido clonado previamente y se desea desplegar la última versión, se deben traer los cambios de la rama principal:

```
cd restify
git pull origin main
```

Una vez ejecutado este paso, se creará la estructura de directorios del proyecto:

```
restify/
├── backend/          # Código fuente de Spring Boot & Dockerfile
├── frontend/        # Código fuente de React (Vite) & Dockerfile
├── certs/           # Carpeta para certificados SSL (Opcional)
├── .env             # Archivo general de variables (A crear)
└── docker-compose.yml # Orquestador de los contenedores
```

2. CREAR EL ARCHIVO DE VARIABLES DE ENTORNO

Una vez dentro de la raíz del proyecto `restify/`, se debe crear un archivo llamado `.env`. Este archivo centraliza las variables para todos los servicios (frontend, backend y base de datos). El contenido debe seguir esta estructura:

```
DB_NAME=db_name
DB_USERNAME=db_username
DB_PASSWORD=db_password
DB_ROOT_PASSWORD=db_root_password

JWT_SECRET=jwt_secret

INITIAL_ADMIN_USERNAME=initial_admin_username
INITIAL_ADMIN_PASSWORD=initial_admin_password
INITIAL_ADMIN_EMAIL=initial@admin.email

MAPTILER_KEY=maptiler_key
```

3. CONFIGURAR LOS CERTIFICADOS SSL (Opcional para Local)

La gestión de certificados varía según el entorno de despliegue:

- **Despliegue en Local** (localhost): Este paso es opcional. Si no se dispone de certificados, Nginx funcionará de forma nativa a través del puerto HTTP `80`.
- **Despliegue en Producción** (Dominio real): Es obligatorio introducir los certificados SSL emitidos para el dominio en la carpeta `./certs/`. Los archivos deben nombrarse estrictamente de la siguiente manera:
 - `restify.crt`
 - `restify.key`

4. LIMPIAR EL ENTORNO DOCKER (Recomendado)

Para asegurar una instalación limpia y evitar conflictos con residuos de ejecuciones anteriores, se aconseja ejecutar:

```
docker-compose down --remove-orphans  
docker system prune -f
```

5. CONSTRUCCIÓN Y LANZAMIENTO

Para compilar el código de los submódulos y levantar la infraestructura completa en segundo plano, se ejecuta el siguiente comando:

```
docker-compose up -d --build
```

3. VERIFICACIÓN DEL ESTADO

Para comprobar que todos los componentes de Restify están operando correctamente, se ejecuta el comando:

```
docker-compose ps
```

Se debe verificar que los tres contenedores principales se encuentran en estado Up:

1. `restify-frontend` (Escuchando peticiones externas en los puertos `80` y `443`).
2. `restify-backend` (Operando en la red interna en el puerto `8080`).
3. `restify-database` (Operando en la red interna en el puerto `3306`).

PRUEBA DE DISPONIBILIDAD

Si se han realizado correctamente los pasos anteriores, la página web debería estar disponible en <http://localhost:80>.

El correcto funcionamiento del proxy inverso se puede validar enviando una solicitud de autenticación desde la terminal:

```
curl -k -i -X POST https://localhost/api/v1/auth/login \  
-H "Content-Type: application/json" \  
-d '{"username":"admin", "password":"password"}'
```

La recepción de un encabezado `HTTP/1.1 200 OK` (o un error de credenciales devuelto por Spring Boot) confirmará que el flujo de red se ha completado con éxito.